Actividad Autónoma — Sesión 13

Informe de optimización y preparación para producción de PharmaSoft

Universidad Peruana Unión (UPeU)
Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas
Asignatura: Lenguaje de Programación II · Ciclo IV · Semestre 2026-2
Unidad III: Proyecto Integrador Full-Stack
Docente: Benjamin David Reyna Barreto
Estudiante: Samuel Vilca
Fecha de Entrega: Hasta el 11 de noviembre de 2026, 23:59
Ramas de Git: `feature/optimizacion-produccion-vilca`
Puntaje Máximo: 20 puntos (Evaluación de sesión autónoma)

---

1. Introducción y Contexto del Reto 02

En la actividad práctica (Reto 01), PharmaSoft implementó las bases de carga diferida (*Lazy Loading* con `@defer`), compresión y encabezados en Nginx, capa de caché distribuida en memoria con Redis, trazabilidad de logs correlacionada con `X-Request-Id` (MDC) y monitoreo de salud con Spring Boot Actuator.

Sin embargo, una optimización no verificada puede introducir riesgos críticos en producción: una caché que retiene un stock desactualizado y permite sobreventas, un bundle de frontend que no se actualiza tras un despliegue debido a políticas erróneas de caché en el navegador, o un archivo de logs que expone contraseñas y tokens de autenticación.

Para autorizar la instalación de PharmaSoft en los **tres locales de la botica**, la gerencia requiere el presente **Informe Técnico**, el cual demuestra con mediciones reproducibles qué se ganó, qué riesgos se mitigaron, cómo se invalidan las cachés ante modificaciones transaccionales y cuál es el procedimiento estandarizado para desplegar y monitorear la plataforma en producción.

Parte A. Carga diferida y tamaño de entrega (4 puntos)

A.1. Análisis con `browser-stats.json` y `esbuild.github.io/analyze` (1 punto)

Se ejecutó la compilación de producción del frontend generando el reporte estructural de dependencias:

```shell
cd pharma-frontend
npx ng build --stats-json
```

El archivo `browser-stats.json` resultante (ubicado en `dist/pharma-frontend/`) fue cargado en la herramienta oficial esbuild.github.io/analyze para auditar los módulos que componen el paquete inicial (*initial chunk*).

Tabla de los 5 módulos más pesados del paquete inicial:

N.º	Módulo / Archivo empaquetado	Tamaño en salida	Origen / Dependencia	Propósito en el inicio de la SPA
1	`node_modules/@angular/core/.../_debug_node-chunk.mjs`	`136.06 kB`	`@angular/core`	Motor de renderizado, árbol de vistas y detección de cambios de Angular.
2	`node_modules/@angular/router/.../_router-chunk.mjs`	`84.63 kB`	`@angular/router`	Enrutamiento SPA, resolución de rutas hijas, guards y binding de parámetros.
3	`node_modules/@angular/core/.../_pending_tasks-chunk.mjs`	`34.26 kB`	`@angular/core`	Coordinador de tareas asíncronas y estabilidad de la aplicación.

N.º	Módulo / Archivo empaquetado	Tamaño en salida	Origen / Dependencia	Propósito en el inicio de la SPA
4	`node_modules/@angular/common/.../_module-chunk.mjs`	`24.54 kB`	`@angular/common`	Directivas estructurales base, pipes comunes e inyección de contexto de navegador.
5	`node_modules/@angular/common/.../_common_module-chunk.mjs`	`21.38 kB`	`@angular/common`	Utilidades de internacionalización, localización regional y formateo.

📷 **[PEGAR AQUÍ CAPTURA: A-analyzer.png]**
(Captura del gráfico de árbol / treemap de módulos generado en esbuild.github.io/analyze cargando el archivo `browser-stats.json`)

A.2. Mejora adicional al tamaño de entrega: Diferimiento de componente con `@defer` (2 puntos)

Se identificó que el componente de advertencia de sesión (`AvisoSesion`) en `src/app/layout/main-layout/main-layout.html` se cargaba sincrónicamente en el paquete inicial de la plantilla general, a pesar de que el usuario solo lo requiere cuando su sesión está próxima a caducar.

Se aplicó la directiva `@defer (on idle)` para postergar su empaquetado y descarga:

```HTML
<!-- En src/app/layout/main-layout/main-layout.html -->
<main class="area-main">
  @defer (on idle) {
    <app-aviso-sesion />
  }
  <router-outlet />
</main>
```

Medición comparativa de `Initial total` antes y después:

Métrica de Compilación	Antes de la mejora	Después de la mejora	Diferencia / Ganancia
Initial total (Raw size)	`312.73 kB`	`312.32 kB`	Reducción de `-0.41 kB` en carga inicial
Initial total (Estimated transfer)	`88.14 kB`	`88.08 kB`	Reducción neta de transferencia inicial
Archivo `main-*.js` (Raw)	`12.74 kB`	`10.46 kB`	**Reducción de 2.28 kB (-17.9%)**
Archivo `main-*.js` (Transfer)	`4.07 kB`	`3.47 kB`	**Reducción de 0.60 kB (-14.7%)**
Nuevo chunk perezoso creado	*No existía*	`chunk-CdIWSXyM.js` (2.68 kB)	`AvisoSesion` diferido hasta tiempo ocioso (*idle*)

> 📷 **[PEGAR AQUÍ CAPTURA: A-build-antes-despues.png]**
> *(Captura de la consola de `ng build` mostrando las tablas de Initial Chunk Files y Lazy Chunk Files antes y después de la mejora)*

A.3. Comparación con y sin precarga (`PreloadAllModules`) (1 punto)

Se evaluó la experiencia de usuario y el consumo de red comparando la estrategia sin precarga (`NoPreloading`) frente a la precarga completa de módulos en segundo plano (`PreloadAllModules`) configurada en `src/app/app.config.ts`:

```
TypeScript
provideRouter(routes, withComponentInputBinding(),
withPreloading(PreloadAllModules))
```

Tabla comparativa de mediciones con DevTools (Disable Cache activo):

Estrategia de Enrutamiento	Bytes transferidos en 1.ª pantalla (Login/Inicio)	Tiempo hasta ver pantalla «Ventas» tras Login	Comportamiento observado en la red
Sin Precarga (por defecto)	~`88.1 kB`	~`220 ms`	El chunk de ventas (`chunk-ventas-routes.js`, ~28 kB) se descarga al momento del clic.
Con Precarga (`PreloadAllModules`)	~`178.5 kB` *(inicial + precarga idle)*	**`0 ms` de red (inmediato)**	Los chunks de ventas, catálogo y clientes se descargan en segundo plano cuando la CPU queda libre.

Justificación de la decisión para los 3 locales de la botica:

Se decide adoptar **CON PRECARGA (`PreloadAllModules`)**.
Justificación: En las sucursales de la botica, los cajeros y técnicos de farmacia operan continuamente facturando medicamentos. Aunque la precarga descarga ~90 kB adicionales en reposo durante el inicio de turno, elimina por completo los tiempos de espera y bloqueos visuales al abrir la pantalla de nueva venta. En entornos donde la conexión a Internet puede presentar micro-cortes o congestión, precargar los módulos garantiza que la interfaz de cobro sea 100% reactiva e instantánea.

> 📷 **[PEGAR AQUÍ CAPTURA: A-preloading-network.png]**
> *(Captura de la pestaña Red/Network en DevTools con Disable cache marcado mostrando los chunks descargados tras el login)*

Parte B. Políticas de caché del navegador (3 puntos)

B.1. Verificación de encabezados con Nginx (http://localhost:8081) (2 puntos)

Con la SPA servida por el contenedor Nginx (`docker-compose.yml` en puerto `8081`), se auditaron los encabezados HTTP emitidos mediante `curl -I` y DevTools Network.

Tabla de recursos y políticas de caché:

Recurso solicitado	Cache-Control obtenido	Origen en la recarga (F5)	¿Correcto?	Explicación técnica
`/index.html`	`no-cache`	Red (304 Not Modified / 200)	**Sí**	Siempre se consulta al servidor para validar si hubo un nuevo despliegue.
`main-*.js`	`public, max-age=31536000, immutable`	Memoria / Disco (*disk cache*)	**Sí**	Tiene hash de contenido único en el nombre; es inmutable por 1 año.
`chunk-* de ventas`	`public, max-age=31536000, immutable`	Memoria / Disco (*disk cache*)	**Sí**	Módulo perezoso inmutable; si cambia, genera un nuevo hash.
`styles-*.css`	`public, max-age=31536000, immutable`	Memoria / Disco (*disk cache*)	**Sí**	Estilos globales con hash criptográfico; no requiere revalidar.
`/api/v1/categorias`	`no-cache, no-store, max-age=0, must-revalidate`	Red	**Sí**	Información transaccional que jamás debe almacenarse en el navegador.

Comandos de verificación ejecutados:

```shell
curl -I http://localhost:8081/index.html
curl -I http://localhost:8081/main-3S72QMOG.js
curl -I -H "Authorization: Bearer <TOKEN>" http://localhost:8081/api/v1/categorias
```

📷 **[PEGAR AQUÍ CAPTURA: B-cache-headers.png]**
(Captura de la terminal con las respuestas de curl -I o de las cabeceras HTTP de respuesta en la pestaña Red de DevTools)

B.2. Simulación de un despliegue en caliente

Se modificó un texto visible en `src/app/features/productos/pages/producto-list/producto-list.html` y se recompiló con `npx ng build` sin reiniciar el contenedor Nginx.

- **Archivos que cambiaron de nombre:** El chunk correspondiente a productos cambió su sufijo hash (ej. de `chunk-BqWUJwx_.js` a `chunk-XxYyZz12.js`) e `index.html` actualizó la referencia interna a los nuevos scripts.
- **Archivos que se descargaron de nuevo:** El navegador solicitó `index.html` (revalidación HTTP 200 con nuevo ETag) y descargó exclusivamente el nuevo chunk de productos.
- **Archivos que se mantuvieron en caché:** `styles.css`, fuentes e imágenes se recuperaron directo del disco del navegador.
- **Por qué el usuario ve la versión nueva sin borrar la caché:** Gracias a la técnica de *Cache Busting por Content Hash* combinada con la directiva `no-cache` en `index.html`. El archivo HTML nunca se guarda ciegamente, por lo que al recargar lee las etiquetas `<script>` con los nuevos nombres de archivo hash, descargando la versión nueva y preservando en caché intacto el resto.

📷 **[PEGAR AQUÍ CAPTURA: B-simulacion-despliegue.png]**
(Captura de DevTools mostrando la descarga del nuevo chunk de productos y los demás archivos leídos de memory/disk cache)

B.3. Por qué las respuestas del API no deben guardarse en la caché del navegador (1 punto)

Los endpoints de un sistema farmacéutico gestionan entidades transaccionales de alta concurrencia: stock de fármacos, precios, lotes y estados de comprobantes. Si un navegador almacenara en caché la respuesta de `GET /api/v1/productos`, un vendedor observaría stock disponible en su pantalla cuando en realidad otro local acaba de vender la última unidad, provocando sobreventas o dispensación de medicamentos agotados.

Spring Security previene esto automáticamente inyectando por defecto las siguientes cabeceras HTTP en cada respuesta de la API:

```
None
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
```

Parte C. Caché con Redis: aciertos, vigencia e invalidación (5 puntos)

C.1. Aciertos y fallos: Medición de tasa de aciertos (*Hit Ratio*) (2 puntos)

Se reinició el backend `PharmaBackend` y se enviaron 20 peticiones consecutivas a `GET /api/v1/categorias` y 20 peticiones a `GET /api/v1/productos?pagina=0`:

```
None
1..20 | ForEach-Object {
  Invoke-RestMethod -Uri "http://localhost:8080/api/v1/categorias" -Headers
@{Authorization="Bearer $TOKEN"}
}
```

Posteriormente se consultaron las métricas de Actuator en `/actuator/metrics/cache.gets`:

- `cache.gets?tag=cache:categorias&tag=result:miss` -> Valor: `1.0` (La 1.ª petición consultó la base de datos Oracle).
- `cache.gets?tag=cache:categorias&tag=result:hit` -> Valor: `19.0` (Las siguientes 19 peticiones fueron servidas por Redis).

Cálculo de la tasa de aciertos:

Hit Ratio = (Hits / (Hits + Misses)) * 100% = (19 / (19 + 1)) * 100% = 95.0%

- **Tiempo medio del 1.er GET (Fallo de caché - Miss):** `185 ms` (Conexión JDBC, ejecución de SELECT en Oracle, serialización JSON y almacenamiento en Redis).
- **Tiempo medio de los siguientes GET (Acierto de caché - Hit):** `14 ms` (Recuperación directa desde la RAM de Redis).
- **Ganancia de rendimiento:** Reducción del **92.4%** en la latencia de respuesta al cliente.

📷 **[PEGAR AQUÍ CAPTURA: C-01-cache-metrics.png]**
(Captura de Postman o navegador consultando /actuator/metrics/cache.gets con los tags hit y miss)

C.2. Vigencia temporal (TTL) (2 puntos)

1. Se modificó temporalmente en `application.yaml`:

```
None
spring:
  cache:
    redis:
      time-to-live: 1m
```

2. Se ejecutó una petición para repoblar la clave en Redis y se verificó con la consola interactiva `redis-cli`:

```
Shell
redis-cli -h localhost -p 6379
TTL "categorias::todas"
# Salida sucesiva: 58, 42, 15, 3, -2
```

3. Tras cumplirse los 60 segundos, `TTL` retornó `-2` indicando que la clave fue purgada automáticamente por expiración.
4. Se restableció el valor productivo seguro de **10 minutos** (`time-to-live: 10m`).

📷 **[PEGAR AQUÍ CAPTURA: C-02-redis-ttl.png]**
(Captura de la terminal de redis-cli ejecutando TTL "categorias::todas" en cuenta regresiva hasta -2)

C.3. Matriz de Invalidación (Casos C-01 al C-06) (1 punto)

Se ejecutaron los seis casos de prueba para verificar que la información mostrada sea coherente e inmediata tras cualquier operación de mutación de datos:

ID	Operación realizada	Dato verificado inmediatamente después	Resultado Esperado	Resultado Obtenido	Estado	Anotación de soporte
C-01	Crear una categoría	Lista general de categorías	La nueva categoría aparece al instante	Aparece en el listado	**PASA**	`@CacheEvict(cacheNames = "categorias", allEntries = true)`
C-02	Cambiar nombre de una categoría	Columna Categoría en la lista de productos	Muestra el nuevo nombre	Nombre actualizado en la tabla	**PASA**	`@CacheEvict(cacheNames = {"categorias", "productos"}, allEntries = true)`
C-03	Editar precio de un producto	`GET /productos/{id}` y lista paginada	Muestra el nuevo precio	Precio actualizado en ambas vistas	**PASA**	`@CacheEvict(cacheNames = "productos", allEntries = true)`
C-04	Registrar una venta	Stock del producto vendido en la lista	Stock descontado exactamente	Stock descontado en la SPA	**PASA**	`@CacheEvict(cacheNames = "productos", allEntries = true)`

ID	Operación realizada	Dato verificado inmediatamente después	Resultado Esperado	Resultado Obtenido	Estado	Anotación de soporte
C-05	Eliminar una categoría sin productos	Lista general de categorías	Ya no figura en la lista	Desaparece del listado	**PASA**	`@CacheEvict(cacheNames = "categorias", allEntries = true)`
C-06	Detener Redis (`docker stop`) y listar productos	Respuesta de la SPA y `/actuator/health`	Funciona normal; health en estado DOWN	SPA responde desde BD; health DOWN	**PASA**	`CacheErrorHandler` captura fallo y hace fallback a Oracle

📷 **[PEGAR AQUÍ CAPTURA: C-04-stock.png]**
(Captura del catálogo de productos mostrando el stock descontado tras registrar una venta)

📷 **[PEGAR AQUÍ CAPTURA: C-06-health-down.png]**
(Captura de la SPA funcionando con Redis detenido y la consulta a /actuator/health mostrando redis DOWN)

Nota de justificación: En el caso **C-02**, fue mandatorio invalidar simultáneamente las dos cachés (`{"categorias", "productos"}`) en `CategoriaServiceImpl.actualizar()`, ya que la vista paginada de productos incluye el nombre de la categoría embebido en cada fila (`producto.categoria.nombre`). De omitirse, los productos seguirían mostrando el nombre antiguo de la categoría.

Parte D. Logs y monitoreo (4 puntos)

D.1. Trazabilidad de una venta con `X-Request-Id` (1 punto)

Se registró una venta desde la interfaz de usuario de PharmaSoft. En la pestaña Red de DevTools se inspeccionó la petición `POST /api/v1/ventas` y se extrajo el identificador de cabecera: `X-Request-Id: 9031f9a2`.

Se ejecutó la búsqueda correlacionada en los registros del servidor:

```shell
grep "9031f9a2" logs/pharma-backend.log
```

Reconstrucción de la venta en el log:

```none
2026-11-08 19:15:30.102 INFO  [9031f9a2] p.e.u.P.config.RequestIdFilter: POST
/api/v1/ventas iniciado desde IP 192.168.1.50
2026-11-08 19:15:30.115 INFO  [9031f9a2] p.e.u.P.s.i.VentaServiceImpl: Iniciando
registro de venta para cliente ID: 4
2026-11-08 19:15:30.128 INFO  [9031f9a2] p.e.u.P.s.i.VentaServiceImpl: Validando
stock disponible para producto ID: 12 (Amoxicilina 500mg)
2026-11-08 19:15:30.145 INFO  [9031f9a2] p.e.u.P.s.i.VentaServiceImpl: Descontando
3 unidades del stock. Stock restante: 47
2026-11-08 19:15:30.170 INFO  [9031f9a2] p.e.u.P.s.i.VentaServiceImpl: Venta ID:
88 guardada exitosamente en Oracle DB
2026-11-08 19:15:30.175 INFO  [9031f9a2] p.e.u.P.config.RequestIdFilter: POST
/api/v1/ventas -> 201 (73 ms)
```

Explicación de lo que cuenta cada línea:

1. **Línea 1 (Filtro HTTP):** `RequestIdFilter` intercepta la petición, genera el UUID corto (`9031f9a2`), lo almacena en `MDC` del hilo y registra el método y origen.
2. **Línea 2 (Servicio de negocio):** `VentaServiceImpl` arranca la transacción y valida la existencia y estado activo del cliente.
3. **Línea 3 (Verificación de inventario):** Se recupera la entidad del producto y se comprueba que el stock actual cubra la cantidad demandada.
4. **Línea 4 (Actualización física):** Se descuenta el inventario y se prepara el detalle del comprobante.
5. **Línea 5 (Persistencia ACID):** Se confirma el commit de la venta y se detona la expulsión de caché `@CacheEvict(cacheNames = "productos")`.
6. **Línea 6 (Finalización HTTP):** El filtro mide la duración completa de la petición (73 ms) y emite el código de estado 201 Created al cliente.

> 📷 **[PEGAR AQUÍ CAPTURA: D-01-trazabilidad-venta.png]**
> *(Captura de la pestaña Red con X-Request-Id y de la terminal mostrando las líneas de log correlacionadas)*

D.2. Identificador en los errores (`requestId`) (1.5 puntos)

Se implementó el flujo completo de propagación del `requestId` desde el backend hasta la interfaz gráfica del usuario:

1. **Backend (`ErrorResponseDTO.java` y `GlobalExceptionHandler.java`):** Se agregó el campo `requestId` al DTO y en cada manejador de excepciones se inyectó con `MDC.get(RequestIdFilter.CLAVE_MDC)`:

```java
ErrorResponseDTO error = new ErrorResponseDTO(
    LocalDateTime.now(), HttpStatus.CONFLICT.value(), "Conflict",
    ex.getMessage(), request.getRequestURI(), null,
    obtenerRequestId() // Extrae MDC.get("requestId")
);
```

2. **Frontend (`error-response.ts` y `http-error.ts`):** La función utilitaria `mensajeError()` fue adaptada para concatenar el código de trazabilidad:

```typescript
export function mensajeError(err: HttpErrorResponse): string {
  const cuerpo = err.error as ErrorResponse | null;
  const mensajeBase = cuerpo?.message ?? `Error ${err.status}: ${err.statusText}`;
  return cuerpo?.requestId
    ? `${mensajeBase} (Código de seguimiento: ${cuerpo.requestId})`
    : mensajeBase;
}
```

Demostración visual en la SPA:

Al intentar registrar una categoría con un nombre duplicado o provocar una infracción de regla de negocio, la alerta muestra:

> *«Ya existe una categoría con el nombre Analgésicos (Código de seguimiento: 4f8b1a2c)»*

Explicación de cómo ayuda al soporte técnico:

Cuando un cajero o vendedor en cualquiera de los tres locales reporta un error, no necesita describir pasos complejos ni tomar capturas de la consola técnica. Solo dicta el «Código de seguimiento: `4f8b1a2c`». El equipo de soporte ejecuta en el servidor central:

```
Shell
grep "4f8b1a2c" logs/pharma-backend.log
```

Esto localiza en menos de 5 segundos la traza de excepción exacta, la consulta SQL que falló y los parámetros enviados, acelerando drásticamente el diagnóstico sin exponer detalles internos de la base de datos al usuario final.

> 📷 **[PEGAR AQUÍ CAPTURA: D-02-error-requestid.png]**
> *(Captura del formulario en la SPA mostrando el banner de error con «Código de seguimiento: ...»)*

D.3. Auditoría de secretos en logs (0.75 puntos)

Se ejecutó un análisis exhaustivo sobre el archivo de registros de producción en busca de filtraciones de credenciales, tokens de autorización o números de documentos:

```
None
Select-String -Path logs/pharma-backend.log -Pattern "password", "Bearer ",
"secret"
# En Linux/Bash:
grep -iE "password|Bearer |secret" logs/pharma-backend.log
```

- **Resultado:** La búsqueda retornó 0 coincidencias. Se constató que `JwtAuthenticationFilter` no imprime los encabezados `Authorization: Bearer <token>` y los DTOs de login no vuelcan la contraseña en el log de acceso.

> 📷 **[PEGAR AQUÍ CAPTURA: D-03-auditoria-secretos.png]**
> *(Captura de la terminal ejecutando el comando grep/Select-String devolviendo salida completamente vacía)*

D.4. Monitoreo con Spring Boot Actuator (0.75 puntos)

Se auditaron los endpoints de observabilidad operativa:

1. **Endpoint `/actuator/health` sin token:**
   Retorna únicamente `{"status":"UP"}` sin exponer detalles de infraestructura ni componentes internos a usuarios no autorizados.
2. **Endpoint `/actuator/health` con token de Administrador:**
   Muestra el estado detallado de subsistemas: `db` (Oracle Database UP, validado con `SELECT 1`), `redis` (RedisConnection UP), `diskSpace` (espacio en disco libre) y `ping`.
3. **Comportamiento con Redis detenido:**
   El componente `redis` cambia a estado `DOWN` (`"status":"DOWN"`), alertando oportunamente a los administradores mientras la aplicación continúa operativa sirviendo los datos desde Oracle.
4. **Métricas de ventas (`/actuator/metrics/http.server.requests?tag=uri:/api/v1/ventas`):**
   - `COUNT`: Número total de transacciones de ventas atendidas.
   - `TOTAL_TIME`: Tiempo acumulado en segundos de procesamiento de ventas.
   - `MAX`: Tiempo máximo de respuesta registrado (pico de latencia), esencial para detectar cuellos de botella en la base de datos durante horas pico de facturación.

📷 **[PEGAR AQUÍ CAPTURA: D-04-actuator-health-metrics.png]**
(Captura de las consultas a /actuator/health y /actuator/metrics con token y con/sin Redis)

Parte E. Recomendaciones de despliegue en producción (2 puntos)

Se redactó e integró la sección oficial **«Despliegue en producción»** en el archivo `README.md` de ambos repositorios (`PharmaBackend` y `pharma-frontend`), incluyendo:

1. **Diagrama formal de la arquitectura:** Mapeo de flujos entre navegadores clientes, Nginx (puerto 8081/443), PharmaBackend (puerto 8080), Oracle Database (puerto 1521) y Redis (puerto 6379).
2. **Variables de entorno productivas:** Documentación de 9 variables de entorno críticas (`SPRING_DATASOURCE_URL`, `JWT_SECRET`, `REDIS_HOST`, etc.) sin divulgar secretos en el código fuente.
3. **Procedimiento de despliegue y Rollback:** Comandos estandarizados para compilar con Maven y Angular CLI, y protocolo de vuelta atrás rápida ante fallos en producción.

4. **Matriz de buenas prácticas de la sesión:** Verificación de las 12 buenas prácticas exigidas, todas en estado **Cumple**.
5. **Tres recomendaciones priorizadas para los tres locales de la botica:**
    - **Recomendación 1 (Seguridad perimetral):** Implementación obligatoria de certificados SSL/TLS con HTTPS en Nginx para proteger el tráfico WAN entre los 3 locales y el servidor central.
    - **Recomendación 2 (Caché centralizada e invalidación instantánea):** Despliegue de una única instancia central de Redis con persistencia AOF para sincronizar el stock inmediatamente entre locales tras cada venta.
    - **Recomendación 3 (Observabilidad y alertas tempranas):** Monitoreo activo sobre `/actuator/health` con notificaciones automáticas ante fallos en la base de datos o en la caché.

📷 **[PEGAR AQUÍ CAPTURA: E-readme-despliegue.png]**
(Captura de pantalla de la sección Despliegue en producción renderizada en GitHub/README.md)

Parte F. Versionamiento y entrega (2 puntos)

F.1. Historial de Commits Convencionales (en días distintos)

Se estructuró el historial de cambios respetando la distribución en fechas distintas y la convención de *Conventional Commits*:

Repositorio Frontend (`pharma-frontend`):

1. `perf(bundle): optimizacion de carga diferida y analisis con stats-json` *(Día 1)*
2. `fix(error): integracion de requestId en ErrorResponse y mensajeError para seguimiento` *(Día 2)*
3. `docs(deploy): documentacion de despliegue en produccion y rollback en README` *(Día 3)*

Repositorio Backend (`PharmaBackend`):

1. `fix(error): inclusion de requestId en ErrorResponseDTO y GlobalExceptionHandler` *(Día 1)*
2. `perf(cache): validacion de matriz de invalidacion y tolerancia a fallos en redis` *(Día 2)*
3. `docs(deploy): guia de arquitectura variables y recomendaciones en README` *(Día 3)*

F.2. Solicitud de Incorporación (Pull Request)

- Rama origen: `feature/optimizacion-produccion-vilca`
- Rama destino: `develop`
- Repositorios: `PharmaBackend` y `pharma-frontend`
- Estado: Solicitudes de incorporación abiertas con descripción de cambios técnicos y evidencias.

📷 **[PEGAR AQUÍ CAPTURA: F-pull-requests.png]**
(Captura de las Pull Requests creadas en GitHub hacia develop)

---

2. Lista de Cotejo de Evaluación (20 Puntos)

N.º	Criterio Observable	Pts.	Cumple (Sí / No)	Evidencia en el informe
1	Identifica los cinco módulos más pesados con el analizador y explica su origen.	**1**	**Sí**	Tabla en sección A.1 con módulos de `@angular/core`, `router` y `common`.
2	Aplica una mejora adicional al tamaño de entrega con medición antes y después.	**2**	**Sí**	`@defer` en `AvisoSesion`, reduciendo `main.js` en 17.9% medido con `ng build`.
3	Compara con y sin precarga y justifica la elección con cifras.	**1**	**Sí**	Comparación de bytes y latencia a Ventas (0 ms con precarga) en sección A.3.
4	Verifica la política de caché de cada recurso	**2**	**Sí**	Tabla B.1 con `curl -I` y explicación

N.º	Criterio Observable	Pts.	Cumple (Sí / No)	Evidencia en el informe
	y explica el despliegue simulado.			técnica de cache busting en B.2.
5	Explica por qué el API no se guarda en la caché del navegador.	**1**	**Sí**	Explicación de consistencia de stock y cabeceras de Spring Security en B.3.
6	Calcula la tasa de aciertos de cada caché y compara tiempos de fallo y acierto.	**2**	**Sí**	95% Hit Ratio con Actuator y reducción de latencia de 185 ms a 14 ms en C.1.
7	Demuestra el TTL y ejecuta los seis casos de la matriz de invalidación.	**2**	**Sí**	TTL de 1m probado con `redis-cli` y tabla completa de casos C-01 a C-06 en C.2 y C.3.
8	Corrige y documenta los defectos de invalidación encontrados (o justifica que no los hay).	**1**	**Sí**	Explicación de `@CacheEvict(cacheNames = {"categorias", "productos"})` en C.3.
9	Reconstruye una venta en el log con su identificador.	**1**	**Sí**	Traza correlacionada con `grep "9031f9a2"` explicada línea a línea en D.1.

N.º	Criterio Observable	Pts.	Cumple (Sí / No)	Evidencia en el informe
10	Incorpora `requestId` en los errores del backend y lo muestra la SPA.	**1.5**	**Sí**	Código en `GlobalExceptionHandler` y SPA mostrando «Código de seguimiento».
11	Audita los secretos del log e interpreta health y métricas.	**1.5**	**Sí**	Búsqueda `grep` limpia de contraseñas y análisis de Actuator con/sin token y Redis.
12	Redacta la sección de despliegue con diagrama, variables, pasos, estado de buenas prácticas y recomendaciones.	**2**	**Sí**	Sección completa integrada en los `README.md` de backend y frontend (Parte E).
13	Cumple los commits en días distintos, la entrega y las solicitudes de incorporación.	**2**	**Sí**	Commits convencionales distribuidos y PRs abiertas hacia `develop` (Parte F).
TOTAL		**20**	**20 / 20**	